

Multithreaded Architectures and Programming on Multithreaded Architectures: Never say never

Hung-Wei Tseng

Recap: Tips of programming on modern processors

- Minimize the critical path operations
 - Don't forget about optimizing cache/memory locality first!
 - Memory latencies are still way longer than any arithmetic instruction
 - Can we use arrays/hash tables instead of lists?
 - Branch can be expensive as pipeline get deeper
 - Sorting
 - Loop unrolling
 - Still need to carefully avoid long latency operations (e.g., mod)
- Since processors have multiple functional units — code must be able to exploit instruction-level parallelism
 - Hide as many instructions as possible under the "critical path"
 - Try to use as many different functional units simultaneously as possible
- Modern processors also have accelerated instructions
- Compiler can do fairly go optimizations, but with limitations

Recap: limited ILP in programs

Most of time, we can't fully
utilize the function units

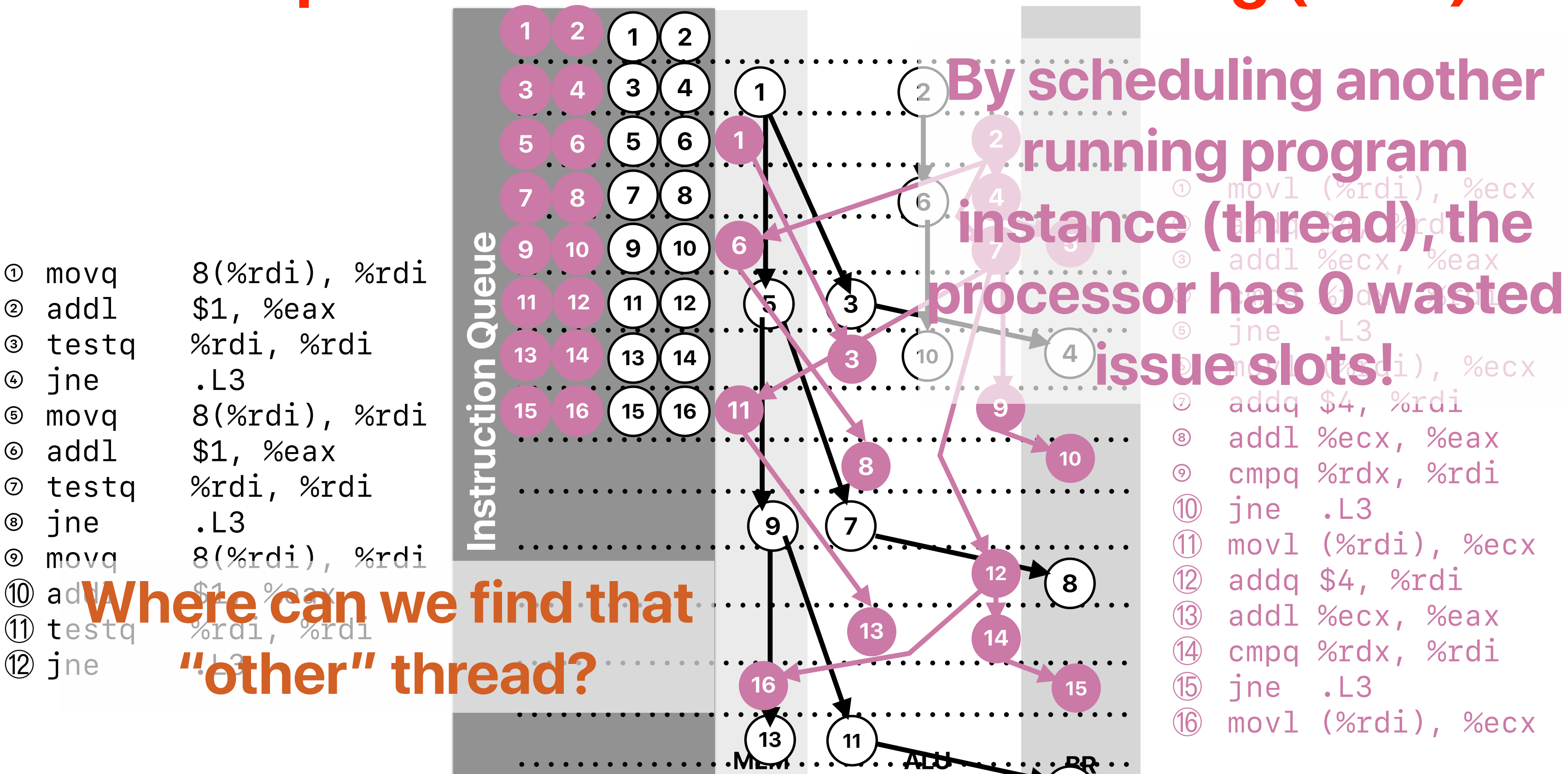
	Cycles	IC	IPC/ILP	ET	# of branches	Branch mis-prediction rate
A	334270949858	118508036769	2.821	23.237	65366730559	0.012
B	290179950840	68341242029	4.246	13.419	18319495534	0.004
C	102882218758	27580097715	3.730	5.380	18129328282	0.004
D	80043714833	19072124536	4.197	3.754	1037120144	0.000
E	253238690876	376641071475	0.672	73.977	73967642413	0.184
SSE4.2	22089754243	8444815558	2.616	1.654	1014543318	0.000

Top at 4.3

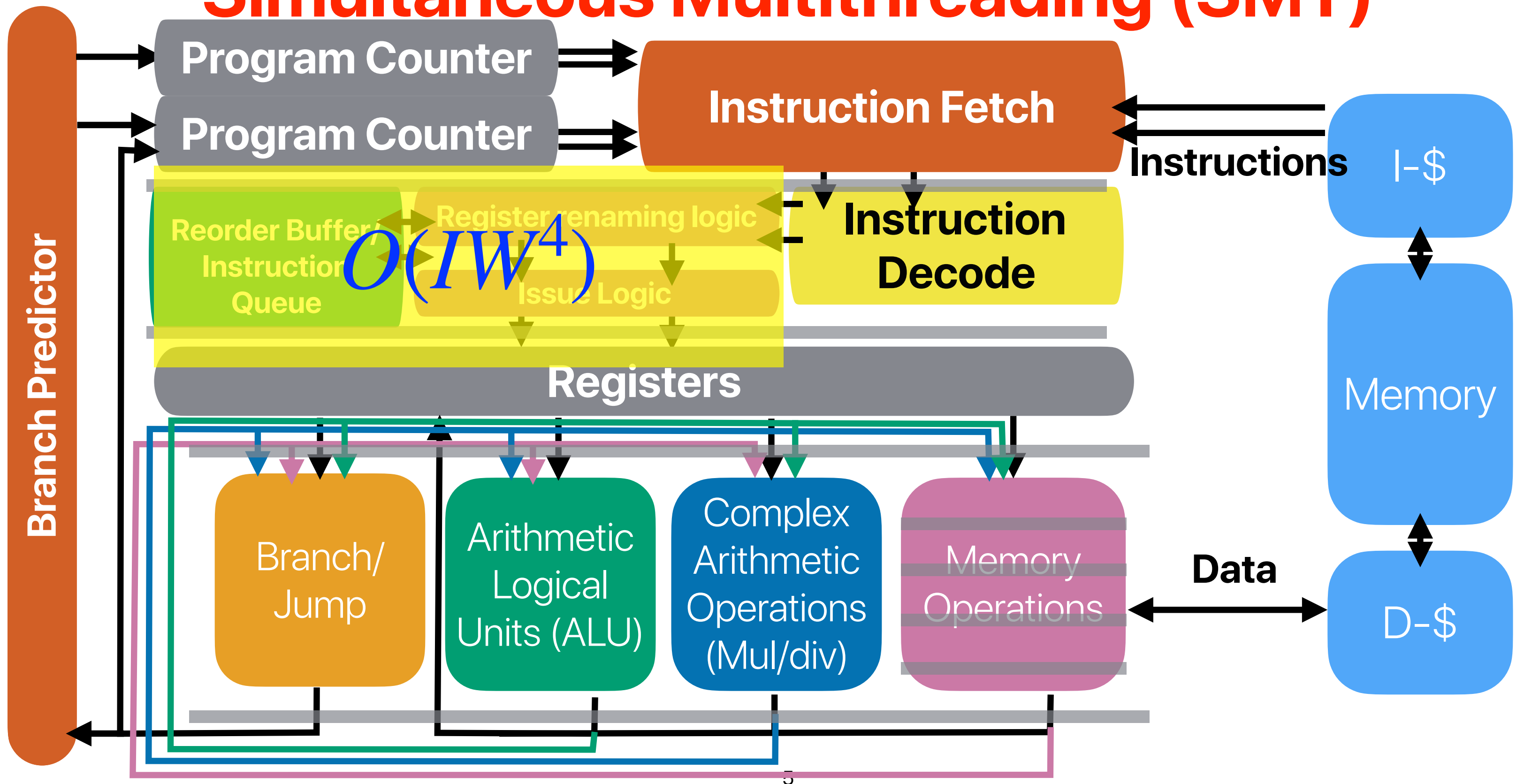
Best performing one at 2.7

Performance code may not
even fully utilize FUs, either.

Concept: Simultaneous Multithreading (SMT)



Simultaneous Multithreading (SMT)



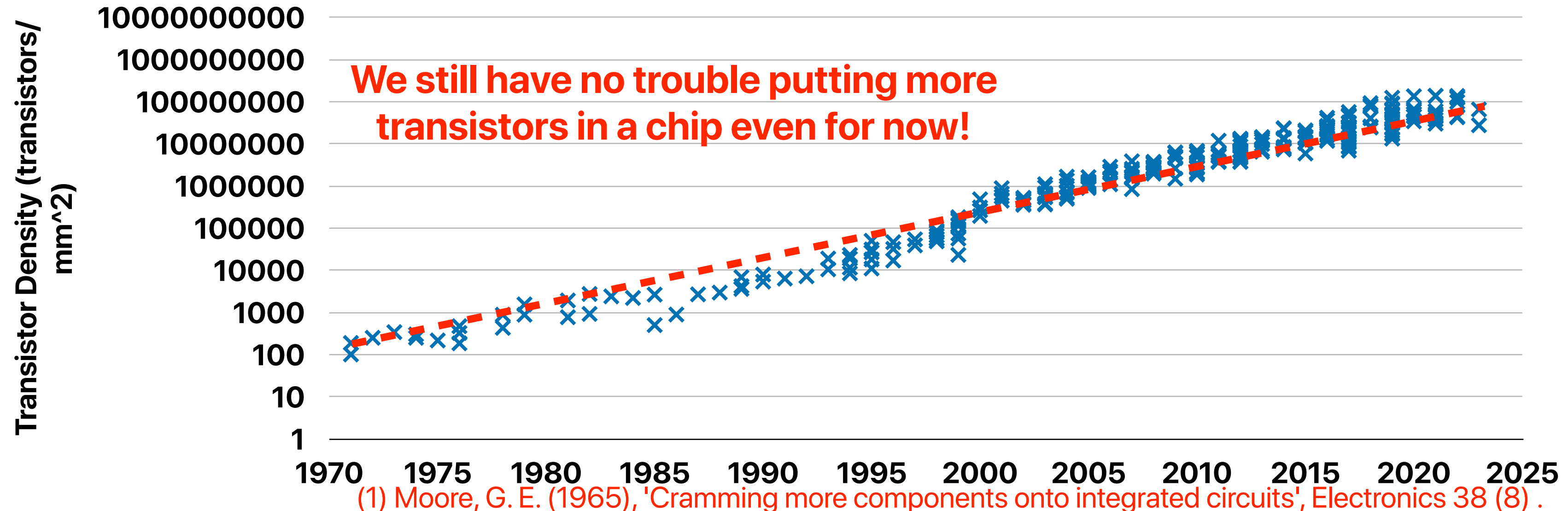
Takeaways: parallel architectures

- SMT processors can better utilize the pipeline resources by allowing simultaneous execution of multiple threads
 - Improved execution throughput
 - May hurt the latency of each thread since we share functional units & cache

Chip-Multiprocessors (CMP) or Multi-core processors

Recap: Moore's Law⁽¹⁾

- The number of transistors we can build in a fixed area of silicon doubles every 12 ~ 24 months.
- Moore's Law "was" the most important driver for historic CPU performance gains



Transistor Counts

Microarchitecture	Transistor Count	Issue-width	Year
Alder Lake	325 M	5x ALU, 7x Memory	2021
Coffee Lake	217 M	4x ALU, 4x Memory	2017
Sandy Bridge	290 M	3x ALU, 3x Memory	2011
Nehalem	182.75 M	3x ALU, 3x Memory	2008



How many transistors per core on Coffee Lake?



The Coffee Lake processor has 217 million transistors per core. It is manufactured using Intel's second 14 nm process. Coffee Lake processors introduced i5 and i7 CPUs featuring six cores (along with hyper-threading in the case of the i7) and no hyperthreading.

The transistor count per core on Coffee Lake is lower than that of some other modern processors, such as the AMD Ryzen 5 5600X. However, Coffee Lake still offers good performance, thanks to its high clock speeds and efficient architecture.

Here is a table of the transistor counts per core for some other modern processors:

Processor	Transistors per core
Coffee Lake	217 million
Ryzen 5 5600X	390 million
Ryzen 7 5800X	550 million

Nehalem 6-issue
Alder Lake 12-issue

Based on https://en.wikipedia.org/wiki/Transistor_count

Recap: do we really need very wide issue processors?

	ET	IC	IPC/ILP	# of branches	Branch mis-prediction rate
A	22.21	332 Trillions	2.88	65 Trillions	1.13%
B	12.29	287 Trillions	4.52	17 Trillions	0.04%
C	5.01	102 Trillions	3.95	17 Trillions	0.04%
D	3.73	80 Trillions	4.13	1 Trillions	~0%
E	54.4	173 Trillions	0.61	44 Trillions	18.6%
SSE4.2	1.57	22 Trillions	2.7	1 Trillions	~0%

One powerful core or two "good enough" cores?

Microarchitecture	Transistor Count	Issue-width	Year
Alder Lake	325 M	5x ALU, 7x Memory	2021
Coffee Lake	217 M	4x ALU, 4x Memory	2017
Sandy Bridge	290 M	3x ALU, 3x Memory	2011
Nehalem	182.75 M	3x ALU, 3x Memory	2008



How many transistors per core on Coffee Lake?



The Coffee Lake processor has 217 million transistors per core. It is manufactured using Intel's second 14 nm process. Coffee Lake processors introduced i5 and i7 CPUs featuring six cores (along with hyper-threading in the case of the i7) and no hyperthreading.

The transistor count per core on Coffee Lake is lower than that of some other modern processors, such as the Ryzen 5 5600X. However, Coffee Lake still offers good performance, thanks to its high clock speeds and efficient architecture.

Here is a table of the transistor counts per core for some other modern processors:

Processor	Transistors per core
Coffee Lake	217 million
Ryzen 5 5600X	390 million
Ryzen 7 5800X	550 million

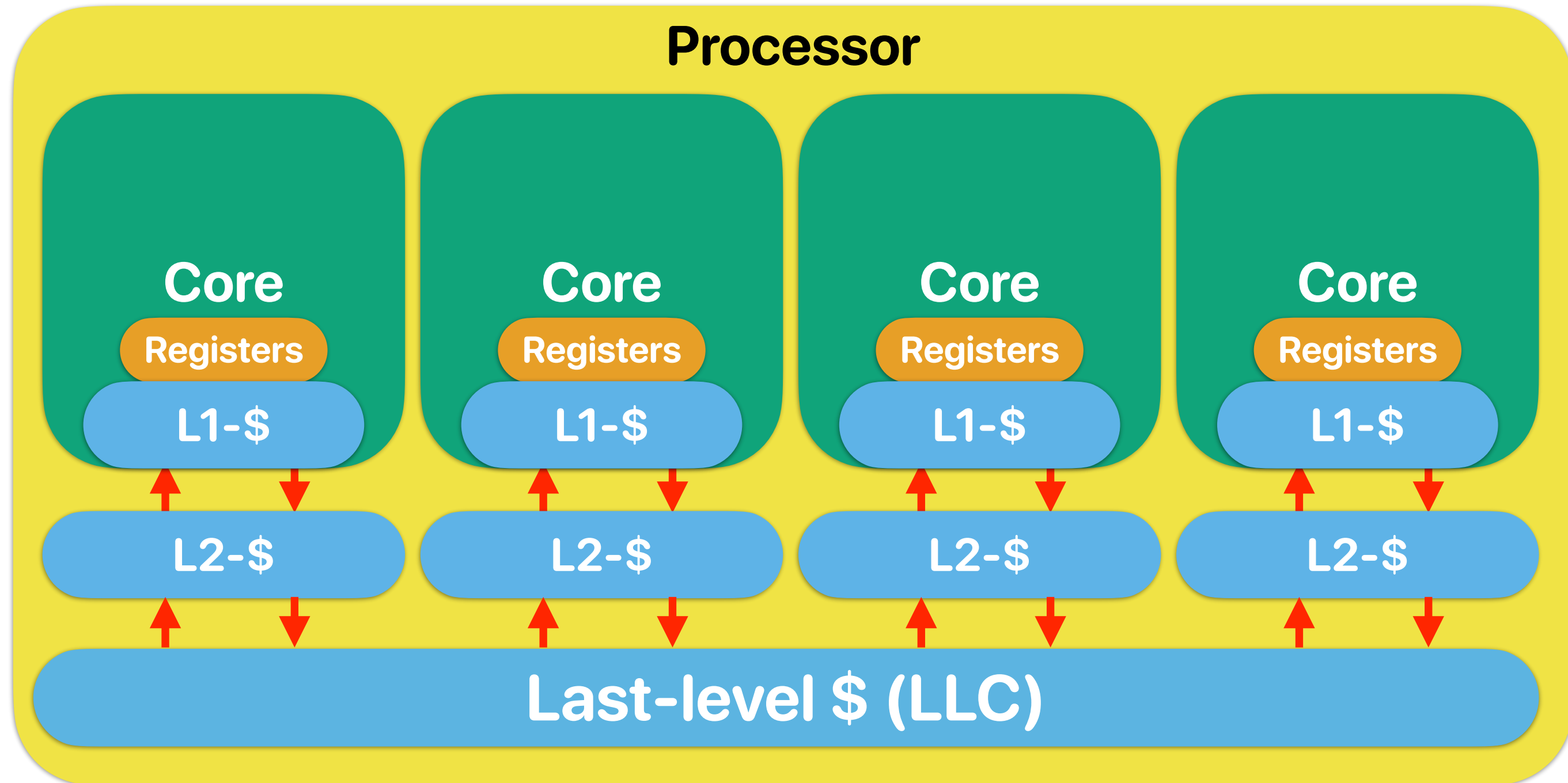
2x 3-issue ALUs Nehalem

Nehalem Alder Lake Nehalem
6-issue 12-issue 6-issue

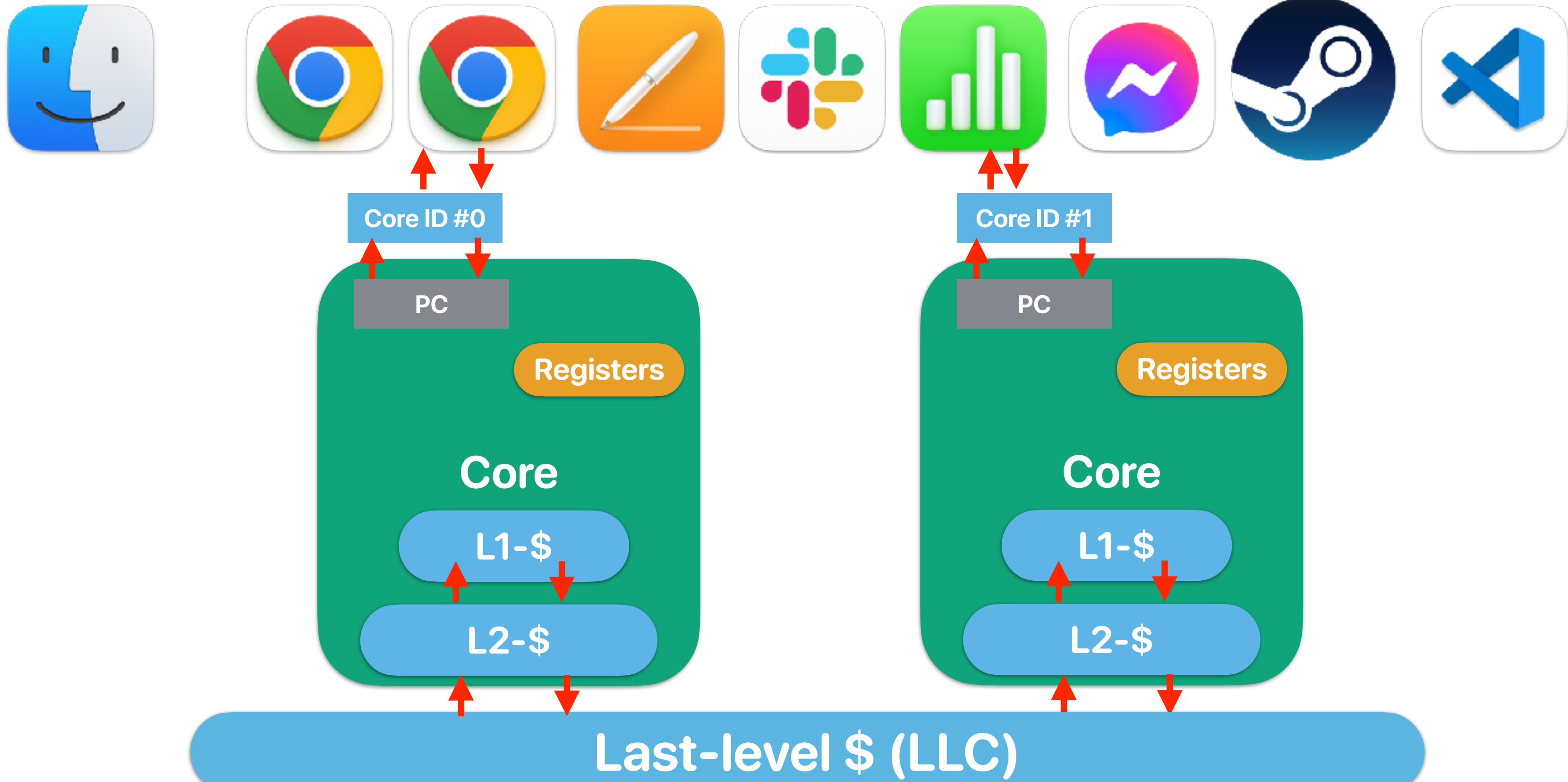
1x 5-issue ALUs Alder Lake

Based on https://en.wikipedia.org/wiki/Transistor_count

Concept of CMP



CMP from the user/OS' perspective





SMT v.s. CMP

- An SMT processor is basically a SuperScalar processor with multiple instruction front-end. Assume within the same chip area, we can build an SMT processor supporting 4 threads, with 6-issue pipeline, 64KB cache or — a CMP with 4x 2-issue pipeline & 16KB cache in each core. Please identify how many of the following statements are/is correct when running programs on these processors.
 - ① If we are just running one program in the system, the program will perform better on an SMT processor
 - ② If we are running 4 applications simultaneously, the cache miss rates will be higher in the SMT processor
 - ③ If we are running 4 applications simultaneously, the branch mis-prediction will be higher in the SMT processor
 - ④ If we are running one program with 4 parallel threads, the cache miss rates will be higher in the SMT processor
 - ⑤ If we are running one program with 4 parallel threads simultaneously, the branch mis-prediction will be longer in the SMT processor

A. 1
B. 2
C. 3
D. 4
E. 5



SMT v.s. CMP

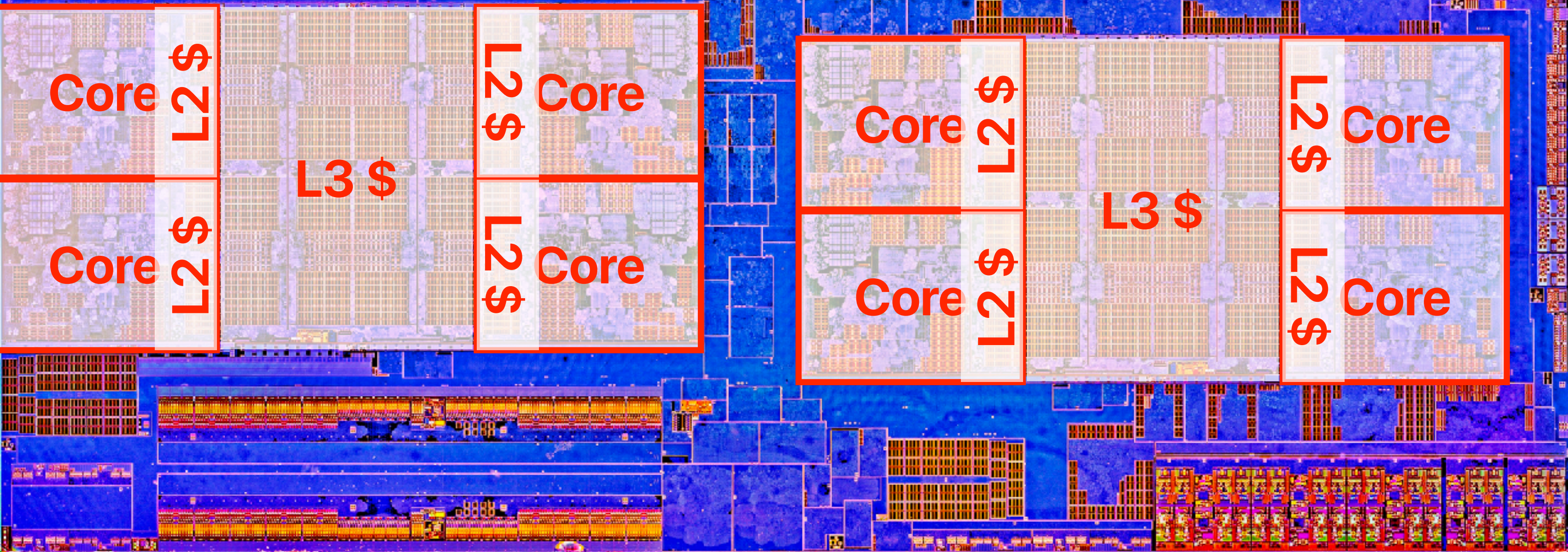
- An SMT processor is basically a SuperScalar processor with multiple instruction front-end. Assume within the same chip area, we can build an SMT processor supporting 4 threads, with 6-issue pipeline, 64KB cache or — a CMP with 4x 2-issue pipeline & 16KB cache in each core. Please identify how many of the following statements are/is correct when running programs on these processors.
 - ① If we are just running one program in the system, the program will perform better on an SMT processor — **you have more resources for the program**
 - ② If we are running 4 applications simultaneously, the cache miss rates will be higher in the SMT processor
 - ③ If we are running 4 applications simultaneously, the branch mis-prediction will be higher in the SMT processor — **it depends!**
 - ④ If we are running one program with 4 parallel threads, the cache miss rates will be higher in the SMT processor — **it depends!**
 - ⑤ If we are running one program with 4 parallel threads simultaneously, the branch mis-prediction will be longer in the SMT processor — **it depends!**
- A. 1 **There is no clear win on each —**
B. 2 **why not having both?**
C. 3
D. 4
E. 5

```
Architecture:                x86_64
CPU op-mode(s):              32-bit, 64-bit
Byte Order:                  Little Endian
Address sizes:               39 bits physical, 48 bits virtual
CPU(s):                      8
On-line CPU(s) list:        0-7
Thread(s) per core:         2
Core(s) per socket:         4
Socket(s):                   1
NUMA node(s):                1
Vendor ID:                   GenuineIntel
CPU family:                   6
Model:                       151
Model name:                  12th Gen Intel(R) Core(TM) i3-12100F
Stepping:                    5
CPU MHz:                     3300.000
CPU max MHz:                 5500.0000
CPU min MHz:                 800.0000
BogoMIPS:                    6604.80
Virtualization:              VT-x
L1d cache:                   192 KiB
L1i cache:                   128 KiB
```

Modern processors have both CMP/SMT



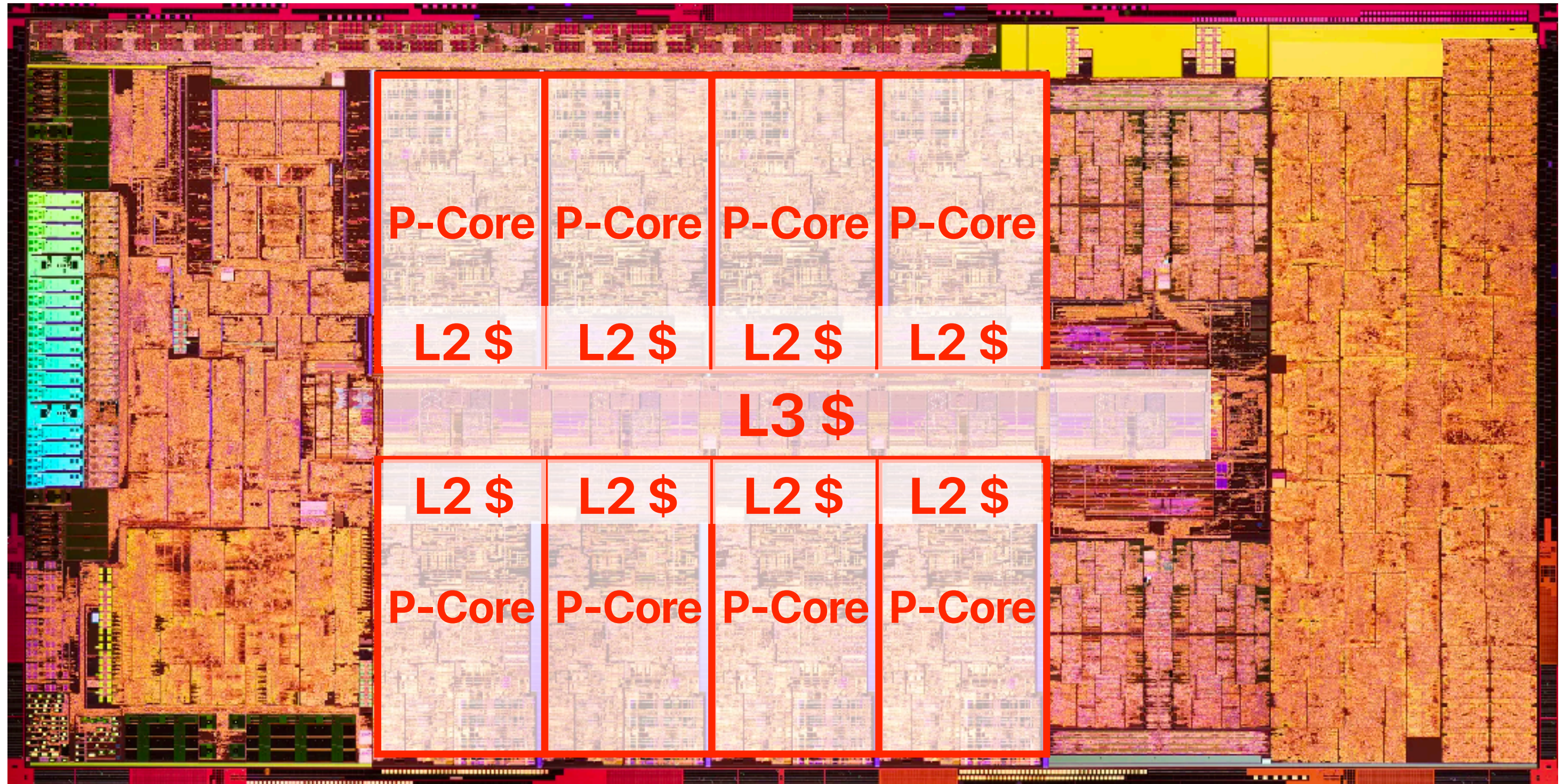
AMD Ryzen Processor



AMD

RYZEN

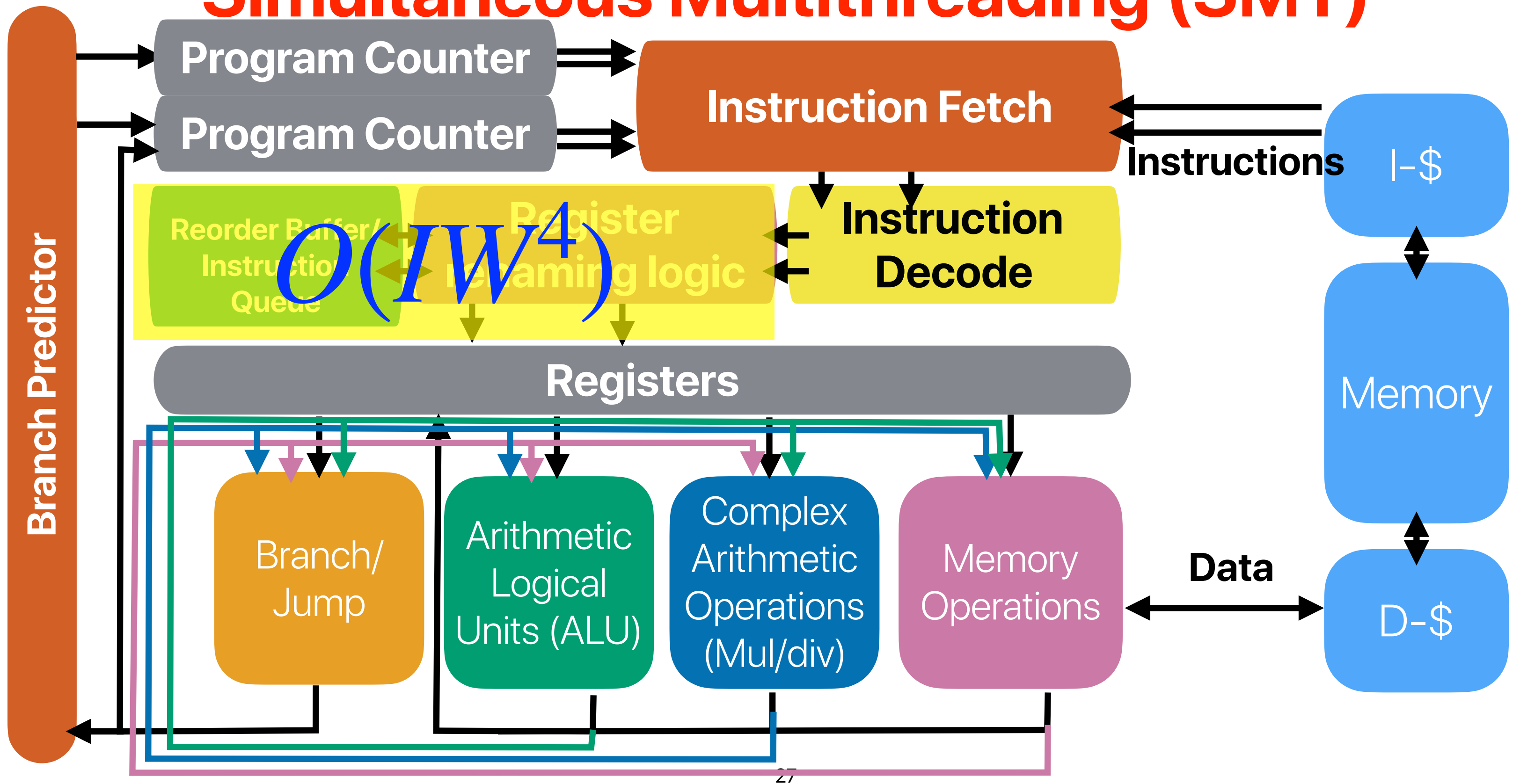
Intel Alder Lake



SMT v.s. CMP

- An SMT processor is basically a SuperScalar processor with multiple instruction front-end. Assume within the same chip area, we can build an SMT processor supporting 4 threads, with 6-issue pipeline, 64KB cache or — a CMP with 4x 2-issue pipeline & 16KB cache in each core. Please identify how many of the following statements are/is correct when running programs on these processors.
 - ① If we are just running one program in the system, the program will perform better on an SMT processor — **you have more resources for the program**
 - ② If we are running 4 applications simultaneously, the cache miss rates will be higher in the SMT processor
 - ③ If we are running 4 applications simultaneously, the branch mis-prediction will be higher in the SMT processor — **it depends!**
 - ④ If we are running one program with 4 parallel threads, the cache miss rates will be higher in the SMT processor — **it depends!**
 - ⑤ If we are running one program with 4 parallel threads simultaneously, the branch mis-prediction will be longer in the SMT processor — **it depends!**
- A. 1 **There is no clear win on each —**
B. 2 **why not having both?**
C. 3
D. 4 **The only thing we know for sure**
E. 5 **— if we don't parallel the program, it won't get any faster on SMT/CMP**

Simultaneous Multithreading (SMT)



Takeaways: parallel architectures

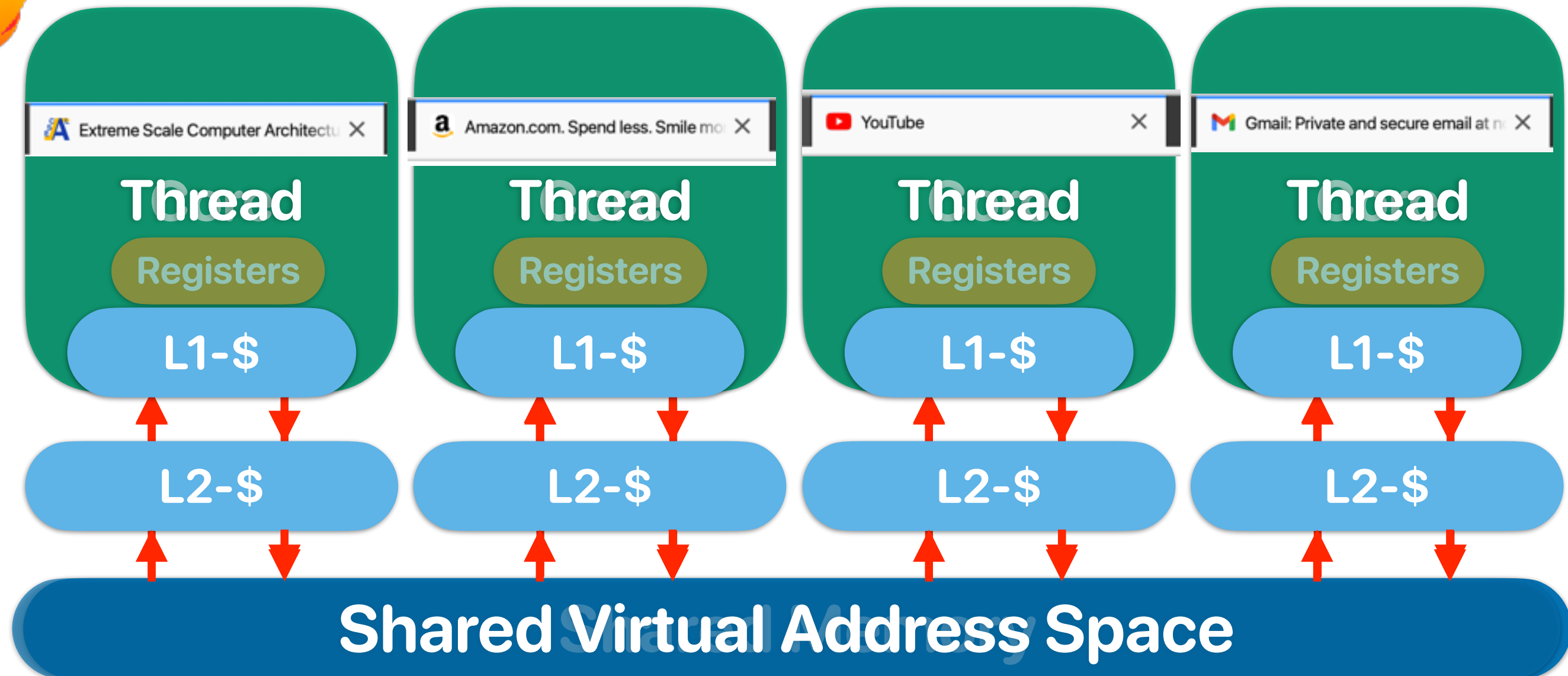
- SMT processors can better utilize the pipeline resources by allowing simultaneous execution of multiple threads
 - Improved execution throughput
 - May hurt the latency of each thread since we share functional units & cache
- CMP provides more processor cores for parallel threads or multiprogrammed environments
 - More isolated, protected pipeline
 - No flexibility if the running program needs more resource

Parallel Programming & Architectural Supports for Parallel Programming

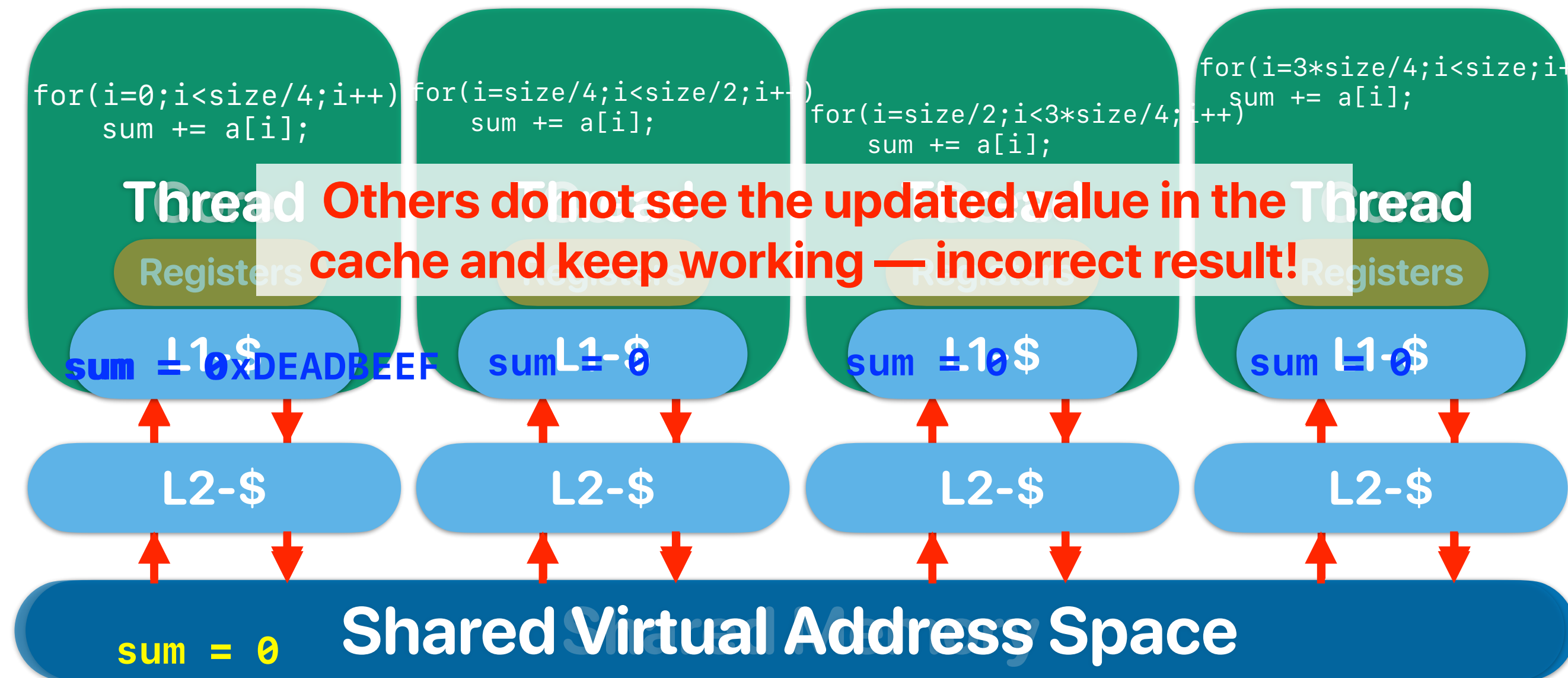
Parallel programming

- To exploit parallelism you need to break your computation into multiple “processes” or multiple “threads”
- Processes (in OS/software systems)
 - Separate programs actually running (not sitting idle) on your computer at the same time.
 - Each process will have its own virtual memory space and you need explicitly exchange data using inter-process communication APIs
 - Programming model: MPI, Spark
- Threads (in OS/software systems)
 - Independent portions of your program that can run in parallel
 - All threads share the same virtual memory space
 - Programming model: pthread or openmp
- We will refer to these collectively as “threads”
 - A typical user system might have 1-8 actively running threads.
 - Servers can have more if needed (the sysadmins will hopefully configure it that way)

What software thinks about "multiprogramming" hardware



What software thinks about "multiprogramming" hardware



Coherency & Consistency

- Coherency — Guarantees all processors see the same value for a variable/memory address in the system when the processors need the value at the same time
 - What value should be seen
- Consistency — All threads see the change of data in the same order
 - When the memory operation should be done

Simple cache coherency protocol

- Snooping protocol
 - Each processor broadcasts / listens to cache misses
- State associate with each block (cacheline)
 - Invalid
 - The data in the current block is invalid
 - Shared
 - The processor can read the data
 - The data may also exist on other processors
 - Exclusive
 - The processor has full permission on the data
 - The processor is the only one that has up-to-date data

Coherent way-associative cache

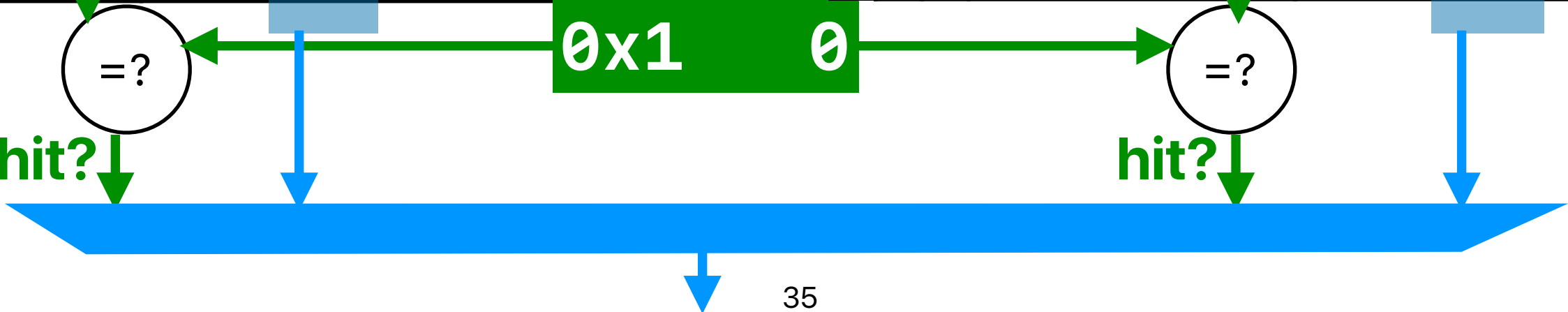
memory address: 0x0

memory address: 0b00001000000100100

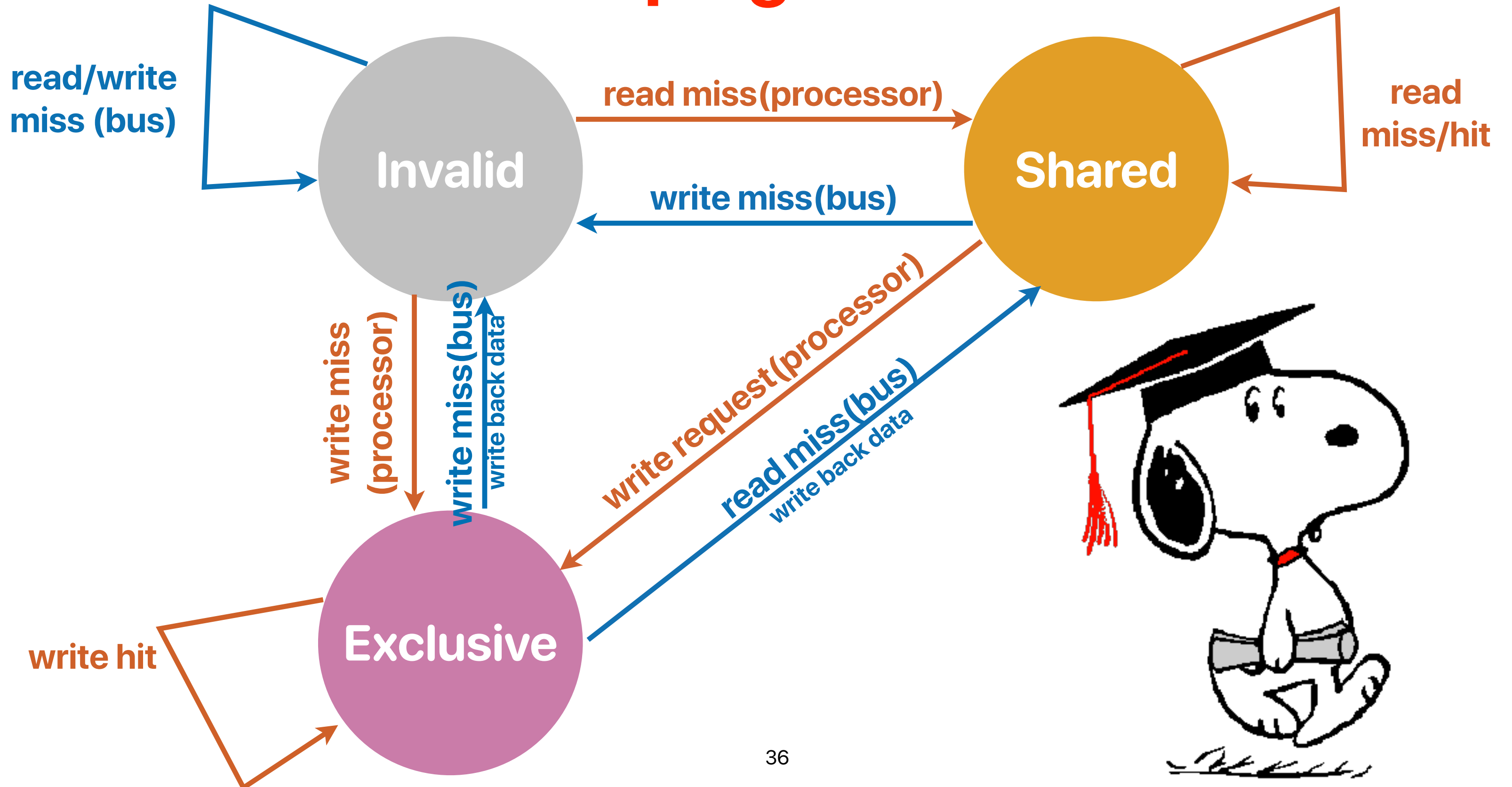
8 tag 2 set 4 block
index offset

States	D	tag	data
01	1	0x29	IIJJKKLLMMNNOOPP
01	1	0xDE	QQRRSSTTUUVVWXX
01	0	0x10	YYZZAABBCCDDEEFF
00	1	0x8A	AABBCCDDEEGGFFHH
10	1	0x60	IIJJKKLLMMNNOOPP
10	1	0x70	QQRRSSTTUUVVWXX
10	1	0x10	QQRRSSTTUUVVWXX
10	1	0x11	YYZZAABBCCDDEEFF

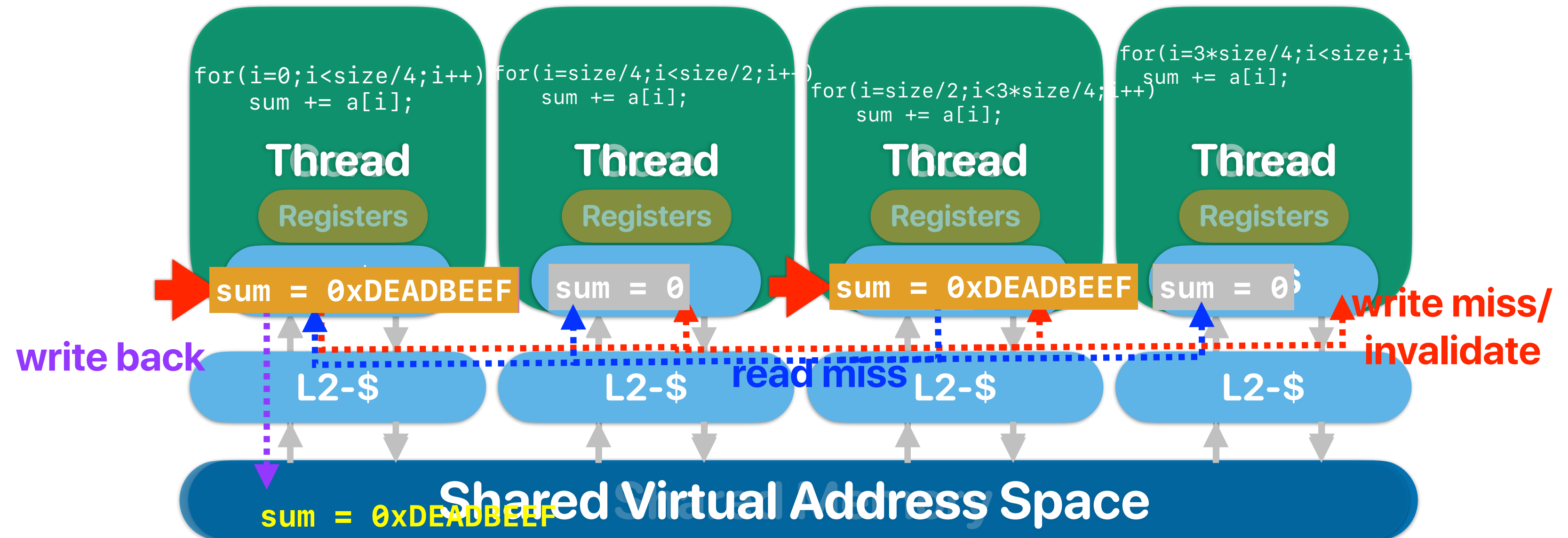
States	D	tag	data
01	1	0x00	AABBCCDDEEGGFFHH
01	1	0x10	IIJJKKLLMMNNOOPP
01	0	0xA1	QQRRSSTTUUVVWXX
00	1	0x10	YYZZAABBCCDDEEFF
10	1	0x31	AABBCCDDEEGGFFHH
10	1	0x45	IIJJKKLLMMNNOOPP
10	1	0x41	QQRRSSTTUUVVWXX
10	1	0x68	YYZZAABBCCDDEEFF



Snooping Protocol



What happens when we write in coherent caches?



Observer

thread 1

```
int loop;

int main()
{
    pthread_t thread;
    loop = 1;

    pthread_create(&thread, NULL,
modifyloop, NULL);
    while(loop == 1)
    {
        continue;
    }
    pthread_join(thread, NULL);
    fprintf(stderr, "User input: %d\n",
```

thread 2

```
void* modifyloop(void *x)
{
    sleep(1);
    printf("Please input a number:\n");
    scanf("%d",&loop);
    return NULL;
}
```

Observer

prevents the compiler from putting the variable "loop" in the "register"

thread 1

thread 2

```
volatile int loop;

int main()
{
    pthread_t thread;
    loop = 1;

    pthread_create(&thread, NULL,
modifyloop, NULL);
    while(loop == 1)
    {
        continue;
    }
    pthread_join(thread, NULL);
    fprintf(stderr, "User input: %d\n",
```

```
void* modifyloop(void *x)
{
    sleep(1);
    printf("Please input a number:\n");
    scanf("%d",&loop);
    return NULL;
}
```



Cache coherency

- Assuming that we are running the following code on a CMP with a cache coherency protocol, how many of the following outputs are possible? (a is initialized to 0 as assume we will output more than 10 numbers)

thread 1	thread 2
<pre>while(1) printf("%d ", a);</pre>	<pre>while(1) a++;</pre>

- ① 0 1 2 3 4 5 6 7 8 9
- ② 1 2 5 9 3 6 8 10 12 13
- ③ 1 1 1 1 1 1 1 64 100
- ④ 1 1 1 1 1 1 1 1 1 100
- A. 0
- B. 1
- C. 2
- D. 3
- E. 4



Cache coherency

- Assuming that we are running the following code on a CMP with a cache coherency protocol, how many of the following outputs are possible? (a is initialized to 0 as assume we will output more than 10 numbers)

thread 1	thread 2
<pre>while(1) printf("%d ", a);</pre>	<pre>while(1) a++;</pre>

- ① 0 1 2 3 4 5 6 7 8 9
 - ② 1 2 5 9 3 6 8 10 12 13
 - ③ 1 1 1 1 1 1 1 64 100
 - ④ 1 1 1 1 1 1 1 1 100
- A. 0
B. 1
C. 2
D. 3
E. 4

Cache coherency

- Assuming that we are running the following code on a CMP with a cache coherency protocol, how many of the following outputs are possible? (a is initialized to 0 as assume we will output more than 10 numbers)

thread 1	thread 2
<pre>while(1) printf("%d ", a);</pre>	<pre>while(1) a++;</pre>

- ① 0 1 2 3 4 5 6 7 8 9
② 1 2 5 9 3 6 8 10 12 13
③ 1 1 1 1 1 1 1 64 100
④ 1 1 1 1 1 1 1 1 1 100
A. 0
B. 1
C. 2
D. 3
E. 4

Takeaways: parallel architectures

- SMT processors can better utilize the pipeline resources by allowing simultaneous execution of multiple threads
 - Improved execution throughput
 - May hurt the latency of each thread since we share functional units & cache
- CMP provides more processor cores for parallel threads or multiprogrammed environments
 - More isolated, protected pipeline
 - No flexibility if the running program needs more resource
- Cache coherence can guarantee that everyone would eventually have a coherent view of data, but not when

Take-aways: parallel programming

- Cache coherency only guarantees that everyone would eventually have a coherent view of data, but not when



Performance comparison

- Comparing implementations of thread_vadd — L and R, please identify which one will be performing better and why

Version L

```
void *threaded_vadd(void *thread_id)
{
    int tid = *(int *)thread_id;
    int i;
    for(i=tid; i<ARRAY_SIZE; i+=NUM_OF_THREADS)
    {
        c[i] = a[i] + b[i];
    }
    return NULL;
}
```

Version R

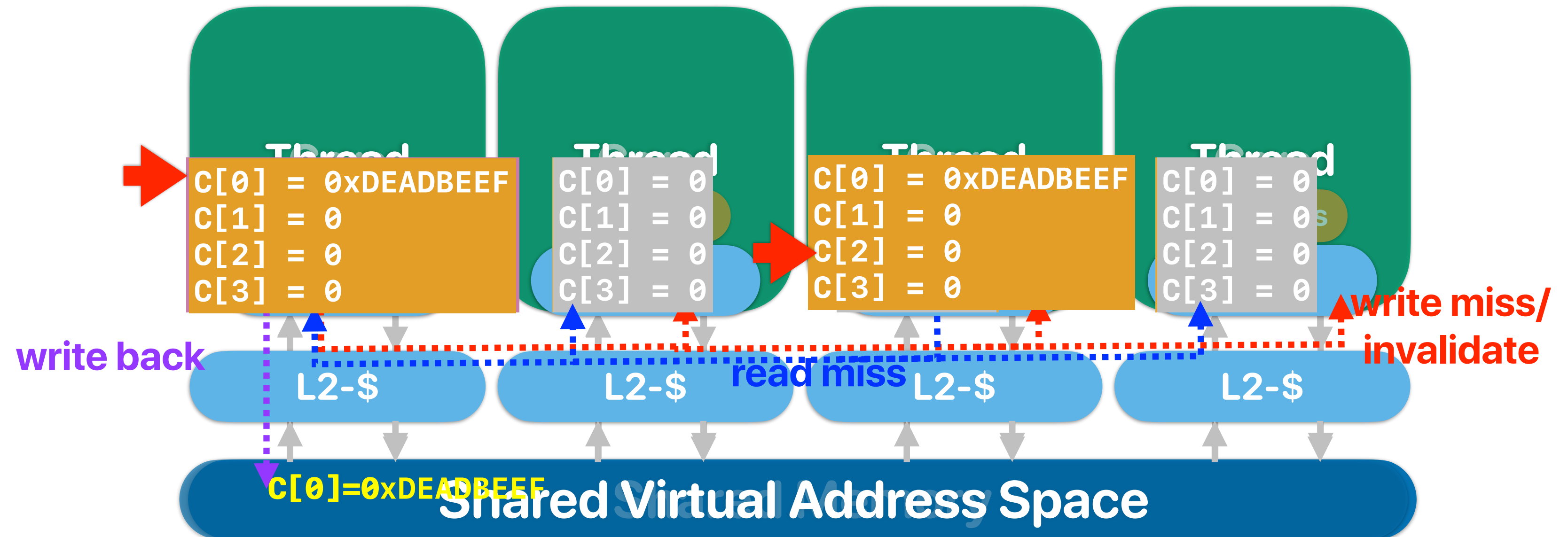
```
void *threaded_vadd(void *thread_id)
{
    int tid = *(int *)thread_id;
    int i;
    for(i=tid*(ARRAY_SIZE/NUM_OF_THREADS); i<(tid+1)*(ARRAY_SIZE/NUM_OF_THREADS); i++)
    {
        c[i] = a[i] + b[i];
    }
    return NULL;
}
```

- A. L is better, because the cache miss rate is lower
- B. R is better, because the cache miss rate is lower
- C. L is better, because the instruction count is lower
- D. R is better, because the instruction count is lower
- E. Both are about the same

Main thread

```
for(i = 0 ; i < NUM_OF_THREADS ; i++)
{
    tids[i] = i;
    pthread_create(&thread[i], NULL, threaded_vadd, &tids[i])
}
for(i = 0 ; i < NUM_OF_THREADS ; i++)
    pthread_join(thread[i], NULL);
```

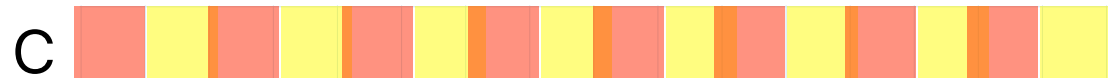
Cache coherency



L v.s. R

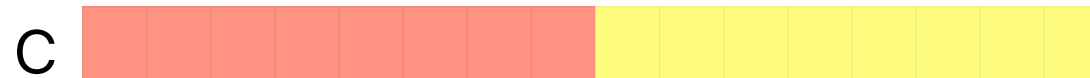
Version L

```
void *threaded_vadd(void *thread_id)
{
    int tid = *(int *)thread_id;
    int i;
    for(i=tid; i<ARRAY_SIZE; i+=NUM_OF_THREADS)
    {
        c[i] = a[i] + b[i];
    }
    return NULL;
}
```



Version R

```
void *threaded_vadd(void *thread_id)
{
    int tid = *(int *)thread_id;
    int i;
    for(i=tid*(ARRAY_SIZE/NUM_OF_THREADS); i<(tid+1)*(ARRAY_SIZE/NUM_OF_THREADS); i++)
    {
        c[i] = a[i] + b[i];
    }
    return NULL;
}
```



4Cs of cache misses

- 3Cs:
 - Compulsory, Conflict, Capacity
- Coherency miss:
 - A "block" invalidated because of the sharing among processors.

False sharing

- True sharing
 - Processor A modifies X, processor B also want to access X.
- False sharing
 - Processor A modifies X, processor B also want to access Y.
However, Y is invalidated because X and Y are in the same block!

Performance comparison

- Comparing implementations of thread_vadd — L and R, please identify which one will be performing better and why

Version L

```
void *threaded_vadd(void *thread_id)
{
    int tid = *(int *)thread_id;
    int i;
    for(i=tid; i<ARRAY_SIZE; i+=NUM_OF_THREADS)
    {
        c[i] = a[i] + b[i];
    }
    return NULL;
}
```

Version R

```
void *threaded_vadd(void *thread_id)
{
    int tid = *(int *)thread_id;
    int i;
    for(i=tid*(ARRAY_SIZE/NUM_OF_THREADS); i<(tid+1)*(ARRAY_SIZE/NUM_OF_THREADS); i++)
    {
        c[i] = a[i] + b[i];
    }
    return NULL;
}
```

- A. L is better, because the cache miss rate is lower
- B. R is better, because the cache miss rate is lower**
- C. L is better, because the instruction count is lower
- D. R is better, because the instruction count is lower
- E. Both are about the same

Main thread

```
for(i = 0 ; i < NUM_OF_THREADS ; i++)
{
    tids[i] = i;
    pthread_create(&thread[i], NULL, threaded_vadd, &tids[i])
}
for(i = 0 ; i < NUM_OF_THREADS ; i++)
    pthread_join(thread[i], NULL);
```

Take-aways: parallel programming

- Cache coherency only guarantees that everyone would eventually have a coherent view of data, but not when
- Cache coherency may create unexpected cache invalidations/misses if you do it wrong



Again — how many values are possible?

- Consider the given program. You can safely assume the caches are coherent. How many of the following outputs will you see?

① (0, 0)

② (0, 1)

③ (1, 0)

④ (1, 1)

A. 0

B. 1

C. 2

D. 3

E. 4

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

volatile int a,b;
volatile int x,y;
volatile int f;
void* modifya(void *z) {
    a=1;
    x=b;
    return NULL;
}
void* modifyb(void *z) {
    b=1;
    y=a;
    return NULL;
}
```

```
int main() {
    int i;
    pthread_t thread[2];
    pthread_create(&thread[0], NULL, modifya, NULL);
    pthread_create(&thread[1], NULL, modifyb, NULL);
    pthread_join(thread[0], NULL);
    pthread_join(thread[1], NULL);
    fprintf(stderr, "(%d, %d)\n", x, y);
    return 0;
}
```



Again — how many values are possible?

- Consider the given program. You can safely assume the caches are coherent. How many of the following outputs will you see?

① (0, 0)

② (0, 1)

③ (1, 0)

④ (1, 1)

A. 0

B. 1

C. 2

D. 3

E. 4

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

volatile int a,b;
volatile int x,y;
volatile int f;
void* modifya(void *z) {
    a=1;
    x=b;
    return NULL;
}
void* modifyb(void *z) {
    b=1;
    y=a;
    return NULL;
}
```

```
int main() {
    int i;
    pthread_t thread[2];
    pthread_create(&thread[0], NULL, modifya, NULL);
    pthread_create(&thread[1], NULL, modifyb, NULL);
    pthread_join(thread[0], NULL);
    pthread_join(thread[1], NULL);
    fprintf(stderr, "(%d, %d)\n", x, y);
    return 0;
}
```

Again — how many values are possible?

- Consider the given program. You can safely assume the caches are coherent. How many of the following outputs will you see?

① (0, 0)

② (0, 1)

③ (1, 0)

④ (1, 1)

A. 0

B. 1

C. 2

D. 3

E. 4

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

volatile int a,b;
volatile int x,y;
volatile int f;
void* modifya(void *z) {
    a=1;
    x=b;
    return NULL;
}
void* modifyb(void *z) {
    b=1;
    y=a;
    return NULL;
}
```

```
int main() {
    int i;
    pthread_t thread[2];
    pthread_create(&thread[0], NULL, modifya, NULL);
    pthread_create(&thread[1], NULL, modifyb, NULL);
    pthread_join(thread[0], NULL);
    pthread_join(thread[1], NULL);
    fprintf(stderr, "(%d, %d)\n", x, y);
    return 0;
}
```

Take-aways: parallel programming

- Cache coherency only guarantees that everyone would eventually have a coherent view of data, but not when
- Cache coherency may create unexpected cache invalidations/misses if you do it wrong
- Processor behaviors are non-deterministic
 - You cannot predict which processor is going faster
 - You cannot predict when OS is going to schedule your thread
 - You cannot predict when the processor is going to schedule an instruction

Announcements

- **Last reading quiz due next Tuesday**
- **Assignment 4 is released** and due this upcoming Sunday

Computer Science & Engineering

142

つづく

